Fit the moment. One change.

Contextual appearance decision engine

You have ten minutes before an interview. You don't need twenty outfit ideas — you need to know whether what you're wearing works, and if not, what single thing to change. Every appearance tool answers *what should I wear?* MIRROR OPS answers **will this look work here?**

THE THESIS, MEASURED

The same look, judged against four different moments. The diagonal is the product — including *overdressed for travel*, which is a mismatch too.

	wedding	interview	dinner	travel
casual	MISMATCH 47	MISMATCH 46	ALMOST 60	ALMOST 69
in between	ALMOST 64	ALMOST 64	FIT 74	ALMOST 67
dressed up	FIT 81	FIT 81	ALMOST 68	ALMOST 60

Contextual fit score 0–100, computed before any change is proposed. A casual look scores 69 for travel and 47 for a wedding.

THE JOURNEY — UNDER 90 SECONDS

```
MOMENT   →  CURRENT LOOK  →  CONTEXTUAL FIT  →  ONE CHANGE  →  PROOF  →  ACT
occasion,   photo +          FIT / ALMOST /     the decisive   YouCam    you
goal, time  YouCam Skin AI   MISMATCH           lever          VTO       leave
```

ONE CHANGE, NEVER A LIST

A single intervention, chosen from a closed space of eight actions. It can **change** a piece, **add** one that is missing, or **remove** one too many — and it always states what stays.

NO CHANGE is a real outcome and consumes no try-on credit. A decision engine that cannot decide to do nothing is a recommendation generator.

WHAT IT IS NOT

Not an AI stylist. Not a shopping assistant. Not a wardrobe manager. Not a try-on app. Not a skin diagnostic tool.

No catalogue to browse, no list to compare, no chatbot. One moment in, one decision out. **The constraint is the product.**

WHERE YOUCAM SITS

Inside the decision loop, not beside it. The browser never talks to the provider; the key stays server-side.

```
               .... face crop, 68% width ....>  SKIN AI      informs the decision
 ONE PHOTO .....:                                            influence capped at 0.08
               '.... original image .........>  APPAREL VTO  proves the decision
                                                             winner only, 1 per journey
```

HOW THE INTEGRATION HOLDS UP

Skin AI needs a close-up; we photograph an outfit.
It rejects images where the face fills under 60% of the width. Asking for a second photo would have added a screen to a 90-second flow, so the server crops the face from the same shot. The original goes untouched to the try-on.

YouCam returns health scores; the engine reasons in severity.
Redness, oiliness and texture are inverted on the way in. Without that, flawless skin would read as heavily marked and push the decision the wrong way.

One journey costs one analysis and one try-on.
Candidates are scored, only the winner is rendered. Enforced by input-aware idempotency keys and by a test that counts provider calls.

A failure never fakes a success.
Offline previews are watermarked and flagged simulated: true all the way to the UI. If Skin AI is unavailable, the journey continues without it, says so, and lowers its stated confidence.

VERIFIED, NOT ASSUMED

324 backend tests · 38 frontend tests · an end-to-end audit of the assembled system.

The audit catches what unit tests cannot: a piece declared absent listed as “kept”, a FIT verdict beside a demanded change, media URLs that return 201 and show nothing.

Both integrations confirmed live on a real full-length photograph: skin metrics returned, and a try-on render with **simulated: false**.

LIMITS WE STATE OURSELVES

On a full-length shot the face is often too small even after cropping, so the skin signal is frequently absent. The journey continues and says so.

There is no user feedback on the decision yet — the one signal that would let the scoring be calibrated on evidence rather than judgement.

We do **not** claim reduced returns. It is a plausible second-order effect, and we have no evidence for it.

RETAIL VALUE

The decision layer is the product; a catalogue is only an input. A retailer's existing product visuals become MIRROR OPS garments through a manifest of `id → URL`, imported in one command with no further integration. That places the try-on at the moment of **hesitation** rather than the moment of browsing: uncertainty → one change → visual proof → decision confidence.

WHAT A JUDGE SHOULD TAKE AWAY

“That AI made a decision for me.” Not: “that app calls a try-on API.” The three moments that carry it: **ALMOST THERE** → **BEFORE / AFTER** → **EVERYTHING ELSE STAYS**

Repository, demo and video links are on the Devpost submission page. Source, tests, deployment guide and the full positioning document ship with the code.